

# FIVE WAYS TO BUILD THE SAME AI CAPABILITY

Lessons from Spring AI, LangChain4j, LangGraph4j, Koog, and Embabel

---

## Abstract

Argonaut implemented the same evidence-grounded AI capability five times on the JVM. Each adapter used the same request, controlled corpus, result envelope, trace semantics, and behavioral contract. Their execution models were nevertheless different: high-level client, declarative service, explicit graph, strategy runtime, and typed goal/action dispatch. This article explains why framework selection is fundamentally a decision about control ownership, not a feature-counting exercise.

## Evidence boundary

All factual claims are bounded to repository-local UC-001 implementations, tests, shared contracts, and engineering reports. This publication does not compare provider quality, latency, cost, popularity, or unexercised framework capabilities.

## Project attribution

Argonaut is a comparative laboratory for JVM AI frameworks. Canonical editable source: <docs/knowledge/articles/UC-ARTICLE-001-five-ways-to-build-the-same-ai-capability.md>

# 1. Why this experiment exists

Argonaut asks a deliberately architectural question: what can we learn by implementing the same controlled AI capability with five fundamentally different JVM abstractions? UC-001 is not a model-quality, token-cost, provider-performance, popularity, or production-resilience benchmark. Its tests use deterministic doubles. The comparison is about how an implementation represents retrieval, state, continuation, model invocation, observability, and its natural testing surface.

## The controlled capability

A question enters a controlled local knowledge repository. The implementation searches and reads documents, selects evidence, synthesizes an answer, and returns an `ExperimentResult` with evidence, trace, metrics, and errors. The same result contract holds even when the execution mechanism changes.

```
public static void verify(ExperimentExecutor executor) {
    assertSatisfied(executor.execute(request()));
}

// Evidence must be traceable to a real document read.
for (Evidence evidence : result.evidence()) {
    assertTrue(tracedSourceIds.contains(evidence.sourceId()),
        "SC4: evidence is not traceable to a repository read");
}
```

**The shared contract is the experiment's guardrail. It proves comparable observable behavior, not equivalent internal topology, efficiency, or answer quality.**

### One observable UC-001 contract

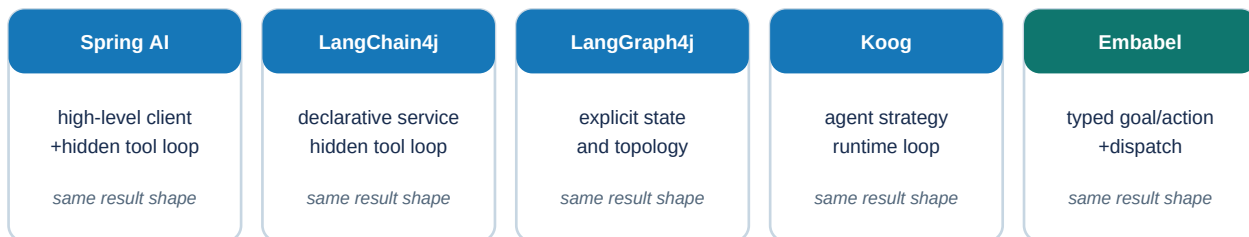


Figure 1. One UC-001 semantic contract, five execution models.

## 2. Spring AI: AI inside a Spring application

The Spring AI adapter is recognizably Spring application code. The application supplies a system prompt, user question, tool, and a per-run context to a high-level `ChatClient`. The client owns the continuation around tool calling.

```
final String answer = chatClient.prompt()
    .system(ControlledLocalEvidencePrompt.SYSTEM_PROMPT)
    .user(request.question())
    .tools(knowledgeTool)
    .toolContext(Map.of("observer", observer, "reads", reads))
    .call()
    .content();
```

`ToolContext` is the material state decision: the tool is Spring-managed, while observer and read accumulator values are owned by a single run. This fits a bounded AI capability embedded in a Spring service. UC-001 also exposes its trade-off: the high-level call hides the tool loop and exact model-call count. The adapter records normalized Argonaut events, but its

reported model count is hard-coded to one rather than measured.

### 3. LangChain4j: a declarative AI service

LangChain4j moves the primary declaration to an annotated Java interface. The builder connects the interface to a model and a per-run tool object; AiServices owns the hidden loop.

```
interface KnowledgeAssistant {
    @SystemMessage(ControlledLocalEvidencePrompt.SYSTEM_PROMPT)
    String answer(String question);
}

var assistant = AiServices.builder(KnowledgeAssistant.class)
    .chatLanguageModel(chatModel).tools(tools).build();
final String answer = assistant.answer(request.question());
```

This makes UC-001 compact. It also makes the inner continuation opaque: the adapter deliberately reports zero model calls as *unknown*, not as a fabricated comparison value. Concision is a real ergonomic advantage, but it is not proof that hidden control is the right model for every workflow.

### 4. LangGraph4j: when flow becomes program

LangGraph4j makes the ReAct shape visible: an agent node calls the model, a conditional edge decides whether tool requests exist, a tools node executes them, and control loops back until the model produces final text.

```
var graph = new MessagesStateGraph<ChatMessage>(serializer)
    .addNode("agent", node_async(state -> {
        var messages = new ArrayList<ChatMessage>();
        messages.add(SystemMessage.from(ControlledLocalEvidencePrompt.SYSTEM_PROMPT));
        messages.addAll(state.messages());
        observer.record(agentEvent(ExecutionEventType.MODEL_CALL_STARTED, Map.of()));
        var response = chatModel.chat(ChatRequest.builder().messages(messages)
            .parameters(ChatRequestParameters.builder().toolSpecifications(toolSpecs).build()).build());
        observer.record(agentEvent(ExecutionEventType.MODEL_CALL_COMPLETED, Map.of()));
        return Map.of("messages", response.aiMessage());
    }))
    .addEdge(START, "agent")
    // actual tool node and conditional edge omitted for space
    .addEdge("tools", "agent")
    .compile();
```

For a short retrieve-and-synthesize path, LangGraph4j is wearing a tuxedo to buy bread. The graph schema, serializer, node actions, and edges make UC-001 look ceremonious next to a high-level client. That is evidence that UC-001 under-exercises graph power, not that explicit topology is a mistake. If evolving state, branches, loops, replayable inspection, or visible transitions are domain requirements, this ceremony can be the point.

### 5. Koog: the loop as strategy

Koog configures an AIAgent, a ToolRegistry, a bounded iteration count, and event handlers. The runtime strategy owns the loop while the adapter can observe model lifecycle events.

```
val toolRegistry = ToolRegistry { tools(knowledgeTools) }
val agent = AIAgent(promptExecutor = executor, llmModel = llmModel,
    toolRegistry = toolRegistry, systemPrompt = ControlledLocalEvidencePrompt.SYSTEM_PROMPT,
    maxIterations = 10) {
    handleEvents {
        onLLMCallStarting { observer.record(agentEvent(MODEL_CALL_STARTED, emptyMap())) }
        onLLMCallCompleted { observer.record(agentEvent(MODEL_CALL_COMPLETED, emptyMap())) }
    }
}
```

Compared with LangGraph4j, the application selects and configures an agent strategy rather than spelling out a topology. UC-001 observed accurate lifecycle-derived model-call counts, but it exercised only the default/simple strategy; it does not benchmark more customized strategy designs.

## 6. Embabel: stop thinking in loops

Embabel changes the framing. The adapter declares an action that can transform an `EvidenceQuestion` into an `EvidenceAnswer`. Its UC-001 retrieval is ordinary deterministic Java inside that action; only answer synthesis uses the LLM.

```
@AchievesGoal(description = "Answer using only controlled local evidence")
@Action
public EvidenceAnswer answer(EvidenceQuestion question, OperationContext context) {
    var searchResponse = knowledgeRepository.search(
        new KnowledgeSearchRequest(question.question(), 5));
    // read selected documents deterministically
    AnswerText answerText = context.ai().withDefaultLlm()
        .createObject(prompt, AnswerText.class);
    return new EvidenceAnswer(finalAnswer, evidences, trace, metrics);
}

EvidenceAnswer answer = AgentInvocation
    .builder(agentPlatform)
    .build(EvidenceAnswer.class)
    .invoke(new EvidenceQuestion(request.runId(), request.question()));
```

The conceptual boundary is: **I have EvidenceQuestion; I want EvidenceAnswer**. That produces a clean action-level test: fake the synthesis context while executing real repository retrieval. It does not demonstrate broader planning behavior. One goal and one action under-exercise the planning and composition questions that make this model distinctive.

### A controlled system need not have a stochastic control plane

**Embabel makes a useful boundary visible: deterministic search -> deterministic document reads -> evidence construction -> stochastic structured synthesis. The lesson is not that deterministic retrieval always wins. It is that LLM discretion and deterministic policy should be placed deliberately, rather than treating every AI-adjacent decision as a tool call.**

## 7. Same contract, completely different execution

| Framework   | Execution ownership      | Retrieval                   | State ownership          | Flow visibility     | Model visibility                   | Natural test unit       | Under-exercised       |
|-------------|--------------------------|-----------------------------|--------------------------|---------------------|------------------------------------|-------------------------|-----------------------|
| Spring AI   | ChatClient internal loop | LLM tool calls              | ToolContext map          | Hidden              | Opaque; 1 hard-coded               | Service + mock model    | Detailed loop control |
| LangChain4j | AiServices internal loop | LLM tool calls              | Per-run tool and service | Hidden              | Opaque; 0 unknown                  | AI service + mock model | Intermediate control  |
| LangGraph4j | Application graph        | Model request; graph routes | Closures and graph       | Explicit            | Direct; accurate                   | Graph + mock model      | State/routing history |
| Koog        | Runtime strategy         | LLM tool calls              | Per-run agent/registry   | Strategy-hidden     | Event-derived; accurate            | Runtime + mock executor | Other strategies      |
| Embabel     | Goal/action dispatch     | Action code                 | Action-local             | Goal/action visible | Call shape known; not hook-counted | Action + fake context   | Multi-action planning |

Table 1. UC-001 comparison. The rows compare observed adapter behavior; values are not a benchmark or a framework ranking.

### Same observable behavior != same execution semantics

Each implementation returns the same core envelope, but control changes hands at different points. Spring AI and LangChain4j hide continuation behind high-level integration. Koog centralizes it in a strategy runtime. LangGraph4j raises topology to an application artifact. Embabel uses goal/action dispatch and may keep retrieval as deterministic application code.

## 8. Who owns the next step?

This is the most useful selection question raised by UC-001. It is not a sophistication ranking. The diagram below maps control placement, from a high-level configured boundary to explicit topology and typed goal/action dispatch.



*Control placement, not a sophistication ranking*

*Figure 2. The models differ in the placement of execution control, not in their legitimacy.*

State ownership follows the same architectural decision. A Spring tool receives context at the client boundary; LangChain4j and Koog construct per-run collaborators; LangGraph4j captures state during graph construction; Embabel keeps it local to an action. These choices determine lifecycle, isolation, inspection, and testability.

## 9. Tool was not the durable abstraction

Four UC-001 adapters expose knowledge access as framework tools. Embabel does not. The durable common concept is instead the capability Argonaut owns:

```
public interface KnowledgeRepository {  
    KnowledgeSearchResponse search(KnowledgeSearchRequest request);  
    DocumentContent read(DocumentReference reference);  
}
```

Tool is an exposure mechanism. KnowledgeRepository expresses stable domain capability. A common abstraction should emerge because the domain needs it, not because several frameworks happen to use similar names. This is why Argonaut shares request/result, evidence, trace, metrics, and knowledge contracts while refusing a universal agent, graph, planner, tool, or model-call abstraction.

## 10. Observability and testability are part of the model

Argonaut records a normalized lifecycle because the experiment needs comparable evidence traceability. Those events are mostly synthetic instrumentation around framework-specific boundaries. Model-call visibility is different: LangGraph4j wraps a direct call; Koog receives runtime lifecycle callbacks; Spring AI and LangChain4j hide the loop in the surface used; Embabel exposes a synthesis call shape but not a hook-counted lifecycle in the action.

Current metric counts must therefore be read with provenance. In UC-001, model calls are exact for LangGraph4j and Koog, hard-coded for Spring AI and Embabel, and unknown for LangChain4j. Tool calls are currently zero in completed metrics even where tools execute. A future metric could carry EXACT, INFERRED, or UNKNOWN provenance; that is a lesson, not a current feature.

Testing surfaces differ too. Spring AI tests a service with a model double; LangChain4j tests an AI service; LangGraph4j tests a graph; Koog tests an agent runtime/strategy; Embabel tests an action with a fake operation context. There is no universal winner. The natural test unit reveals the unit of control the framework asks the application to own.

## 11. Practical selection guidance

| Problem shape                                               | Evaluate first | Trade-off / warning sign                                                    | UC-001 confidence                                 |
|-------------------------------------------------------------|----------------|-----------------------------------------------------------------------------|---------------------------------------------------|
| Bounded Spring capability with prompt and tools             | Spring AI      | Need for exact loop topology or metrics may exceed the high-level boundary  | Moderate for simple tool use                      |
| Concise declarative Java AI service                         | LangChain4j    | Opaque intermediate steps or complex state can make hidden control limiting | Moderate for declarative tool use                 |
| State, branches, loops, and transitions are domain behavior | LangGraph4j    | Straight-line interaction can be obscured by graph machinery                | High for explicit topology; low beyond UC-001     |
| Kotlin agent runtime, bounded strategy, lifecycle events    | Koog           | Only the default/simple strategy was exercised                              | Moderate                                          |
| Typed goals/actions frame deterministic work and LLM use    | Embabel        | Planning is attractive but unexercised in one-action UC-001                 | Moderate for one action; low for broader planning |

**The right selection criterion is contextual: choose the abstraction whose control model resembles the problem. Do not confuse the shortest UC-001 adapter with the best fit for a stateful or multi-responsibility workflow.**

## 12. What UC-001 cannot tell us

UC-001 is mock-backed and deliberately bounded. It cannot compare real-provider quality, cost, latency, streaming, production resilience, multi-turn memory, or checkpoint/resume behavior. It under-exercises LangGraph4j topology, Embabel planning, and Koog alternatives to the default/simple strategy. Its mock sequences make call-efficiency comparisons invalid, and its current metric shape does not provide uniformly exact model/tool counts.

### UC-002 needs a problem worthy of graph and planning abstractions

The next experiment should require evolving state, conditional branching, alternative valid paths, validation, revision or retry, parallel work and join, possibly multiple responsibilities, optional human intervention, and observable topology/provenance. That does not design UC-002. It specifies the missing stress: give explicit graphs, agent strategies, and planners a workflow in which their control model can materially change the outcome.

## Conclusion

**We started by looking for differences between five AI frameworks. What we found were five different answers to a more fundamental architectural question: who should control the next step? None is universally correct. The appropriate abstraction depends on where a system needs determinism, visibility, flexibility, and planning. Implementing the same capability five ways turns framework selection from a feature-counting exercise into a concrete discussion of control ownership.**

**Sources.** Primary evidence: ARGONAUT-MILESTONE-UC001-001 five-framework comparative synthesis; the five repository-local UC-001 agents and tests; ControlledLocalEvidenceContract; ExperimentRequest/Result; KnowledgeRepository; Evidence; ExecutionTrace; and ExecutionMetrics. No external framework research was used.